RK628F/H HDMI-IN 功能开发指南

文件标识：RK-KF-YF-E06

发布版本：V1.2.0

日期：2026-3-20

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

本文档是基于RK3588/RK3576/RK3572等平台，使用RK628F/H开发HDMI IN功能的帮助文档。

概述

RK628F/H支持将HDMI输入信号转换成MIPI-CSI/MIPI-DSI/BT120信号输出，RK3588/RK3576支持接收该信号，实现HDMI-IN功能。本文档仅针对RK628F/H在RK3588/RK3576平台的HDMI-IN功能调试指导，其他场景可按照如下参考其他文档：

RK628F/H显示通路调试可参考：《Rockchip_Developer_Guide_RK628_For_All_Porting_CN》

RK628F/H在RK356X/RK3399等主控调试可参考：
《Rockchip_Developer_Guide_RK628_For_All_Porting_CN》

Android9/10/11版本调试可参考：《Rockchip_Developer_Guide_HDMI_IN_Based_On_CameraHal3_CN》

其他转接芯片Android12+版本调试参考：
《Rockchip_Developer_Guide_Android12+_HDMI_IN_Bridge_CN》

产品版本

芯片名称	内核版本	**Android**版本
RK3588	Linux 5.10/Linux 6.1	Android12/13/14
RK3576	Linux 6.1	Android14
RK3572	Linux 6.12	Android16

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	范建威	2024-8-8	初始版本
V1.1.0	陈顺庆	2024-8-13	补充配置说明
V1.1.1	范建威	2024-9-23	补充BT1120链路配置 补充相关说明以及黑屏问题排查
V1.1.2	陈顺庆	2025-10-29	补充EQ配置说明和HDR配置说明
V1.2.0	范建威	2026-3-20	增加RK3572描述，花屏问题调试说明

目录

RK628F/H HDMI-IN 功能开发指南

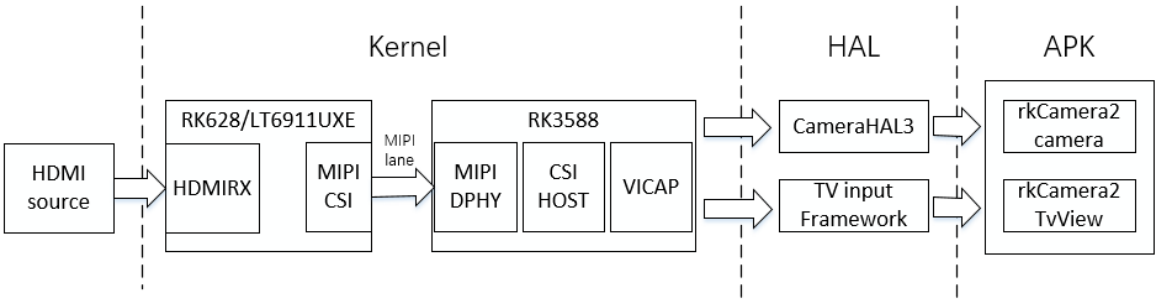
1. HDMI IN功能概述
   - 1.1 RK628F/H 功能框图
   - 1.2 RK SOC接收能力
   - 1.3 RK628F/H 规格简介

2. HDMI IN VIDEO框架
2.1 HDMIIN VIDEO工作流程
2.2 RK628F/H 主要驱动架构
2.3 双MIPI拼接方案
2.4 图像传输延时
3. 关键配置说明
3.1 RK628F/H DTS配置
3.2 Pipeline链路配置
3.2.1 HDMI转MIPI链路
3.2.2 HDMI转BT1120链路
3.3 双MIPI配置
3.4 CSI/DSI模式配置
3.5 scaler 配置
3.6 HDCP
3.7 CEC
3.8 低延时
3.8.1 VI提前送帧机制
3.8.2 使用限制
3.9 色域转换说明
3.10 EQ 配置说明
3.10.1 开启 CED
3.10.2 动态 EQ
3.10.3 固定 EQ
3.11 HDR10
4. HDMIIN APK 适配方法
4.1 APK源码
4.2 TV框架适配
4.3 Camera框架适配
4.3.1 打开宏配置
4.3.2 camera3_profiles.xml配置
4.3.3 切换预览
4.4 TIF预览与camera预览差异
5. RK628F/H 使用一些限制说明
5.1 特殊Timing限制
5.1.1 隔行分辨率
5.1.2 最大分辨率限制
5.1.3 HBP限制
5.1.4 VFP限制
5.1.5 Scaler限制
5.2 CSI/DSI格式说明
5.3 对接RK3588 D/CPHY说明
6. 常用调试方法
6.1 打开log开关
6.2 查询驱动timings信息
6.3 查看media拓扑结构
6.4 查看RK628F/H HDMI-RX状态
6.5 RK628F/H 设置pattern输出
6.6 RK628F/H 寄存器读写
6.7 HDCP调试指令
6.8 开启图像数据流
6.9 抓取图像文件
6.10 TV预览抓图
6.11 查看camera是否注册成功
6.12 camera预览方式抓图
6.13 抓取图层信息
7. 常见问题
7.1 黑屏问题排查
7.2 4096x2160分辨率异常问题

7.3 信号源1440x240黑屏
7.4 1366x768分辨率异常
7.5 应用接口使用说明
7.6 闪花屏问题说明

1. HDMI IN功能概述

1.1 RK628F/H 功能框图



根据应用场景需求，RK628F/H HDMI TO MIPI-CSI2可适配TV框架（后TIF同）或是Camera框架，适配TIF框架图像传输延时更低，适配Camera框架可以使用标准Camera API，更方便录像、对接后端算法等应用功能开发。

1.2 RK SOC接收能力

RK628F/H 类似于SOC Camera设备，接入数量以及接收规格受限于MIPI PHY和DVP接口资源，同时也受限于DDR带宽。

RK3588/RK3576/RK3572 HDMI转MIPI-CSI规格：

芯片型号	**MIPI RX数量**	最大分辨率**(PHY规格)**	支持格式**(MIPI RX)**
RK3588s	**3路：** 3x4lane **4路：** 2x2lane + 2x4lane	DPHY: 1xDPHY-v1.2: 2.5G/lane x 4lane 4K60YUV422 DCPHY: (1)DPHY: 2xDPHY-v2.0: 2.5Gbps/lane x 4lane 4K60YUV422 (2)CPHY: 2xCPHY-v1.1: 2.5Gsps/trio(5.7Gbps) x 3trios 4K60YUV422/RGB888	YUV422/RGB888/RGB565
RK3588	**4路：** 4x4lane **6路：** 4x2lane + 2x4lane	DPHY: 2xDPHY-v1.2: 2.5G/lane x 4lane 4K60YUV422 DCPHY: (1)DPHY: 2xDPHY-v2.0: 2.5G/lane x 4lane 4K60YUV422 (2)CPHY: 2xCPHY-v1.1: 2.5Gsps/trio(5.7Gbps) x 3trios 4K60YUV422/RGB888	YUV422/RGB888/RGB565
RK3576	**3路：** 3x4lane **5路：** 4x2lane + 1x4lane	DPHY: 2xDPHY-v1.2: 2.5G/lane x 4lane 4K60YUV422 DCPHY: (1)DPHY: 2xDPHY-v2.0: 2.5G/lane x 4lane 4K60YUV422 (2)CPHY: 2xCPHY-v1.1: 2.5Gsps/trio(5.7Gbps) x 3trios 4K60YUV422/RGB888	YUV422/RGB888/RGB565
RK3572	**2路：** 2x4lane **4路：** 4x2lane	DPHY: 2xDPHY-v1.2: 2.5G/lane x 4lane 4K60YUV422	YUV422/RGB888/RGB565

注：CPHY描述带宽吞吐率单位为sps，与bps的换算：1 Gsps = 2.28 Gbps。

HDMI To BT1120规格受限于PCLK大小，规格如下：

芯片型号	支持数量	最大分辨率	支持格式	逐行隔行支持
RK3588s	1	双边沿4K30	YUV422	支持逐行/隔行
RK3588	1	双边沿4K30	YUV422	支持逐行/隔行
RK3576	1	双边沿4K30	YUV422	支持逐行/隔行
RK3572	1	双边沿4K30	YUV422	支持逐行/隔行

1.3 RK628F/H 规格简介

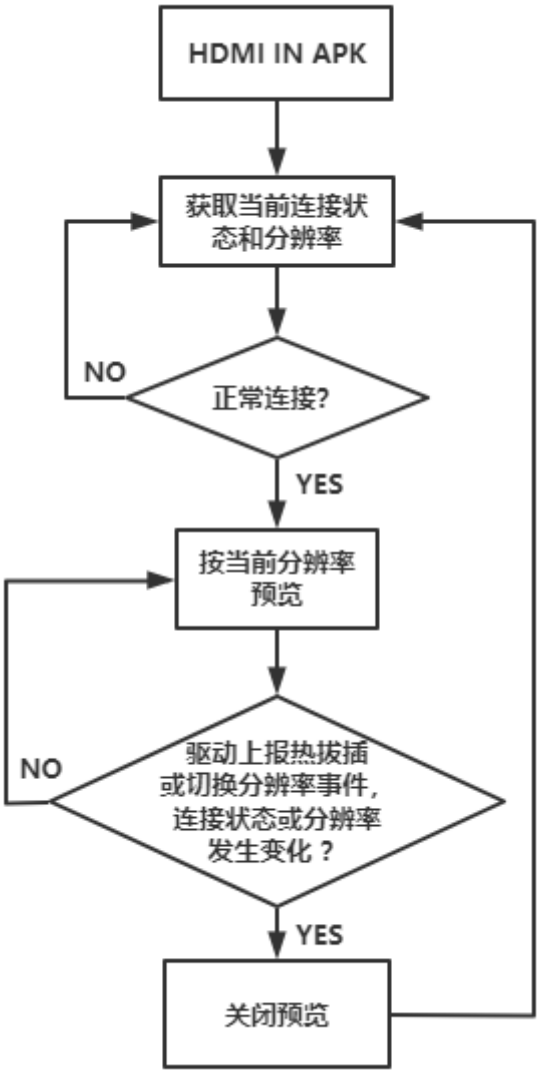
RK628F/H 规格如下表所示：

转接芯片	功能	输入接口	输出接口
RK628F	HDMI2CSI HDMI2DSI HDMI2BT1120	HDMI：4K60 RGB888 / YUV444 / YUV422 / YUV420 / YUV420 10bit	CSI: 双MIPI 4K60/YUV422 DSI: 双MIPI 4K60/RGB888 BT1120: 1080P60/YUV422

2. HDMI IN VIDEO框架

2.1 HDMIIN VIDEO工作流程

HDMI转MIPI-CSI实现HDMIIN功能流程主要如下图所示：

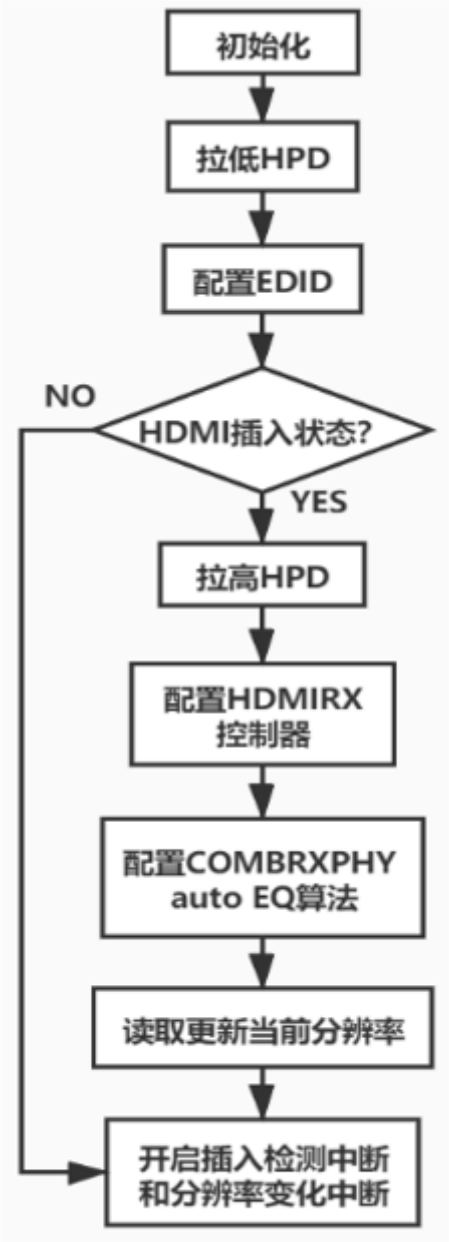


2.2 RK628F/H 主要驱动架构

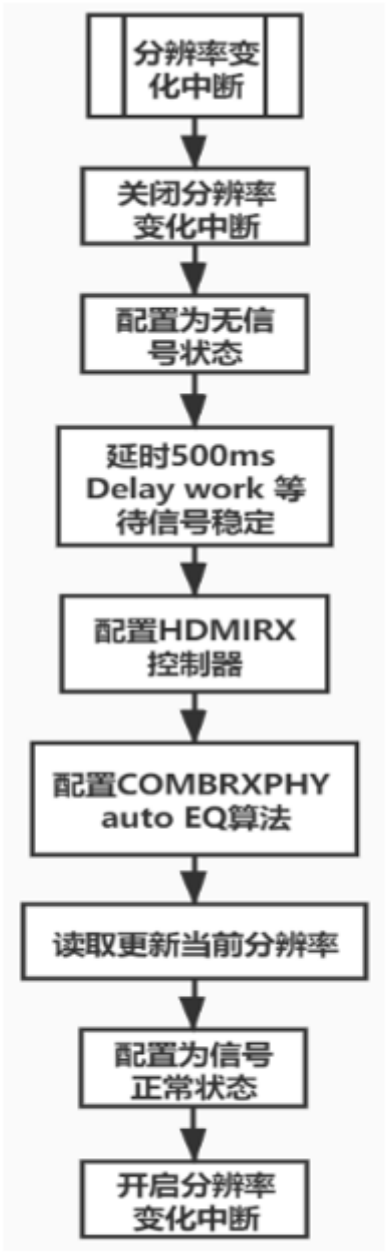
RK628F/H驱动框架基于V4l2框架和media-framework的架构设计实现，并增加了分辨率切换功能以及热拔插功能的设计，基于V4l2/media-framework的驱动框架设计可以参考文档：
《Rockchip_Driver_Guide_VI_CN_v1.1.4.pdf》，这里重点关注分辨率变化与热拔插功能相关的几个中断和delayed_work：

- plugin_detect_irq：5V检测中断，用于检测HDMI接口的拔插动作。
- rk628_csi_irq_handler：HDMI RX中断，监测HDMI信号变化时使用 。
- delayed_work_hotplug：5V检测中断对应的delayed_work，产生热拔插动作时，进行相应的配置处理，并设置拔插状态，上报拔插事件 。
- delayed_work_res_change：HDMI RX控制器检测到信号变化中断时，对转接芯片进行重新配置，获取分辨率，上报分辨率变化事件。
- subscribe_event：注册v4l2事件，主要注册了分辨率变化事件与拔插事件。

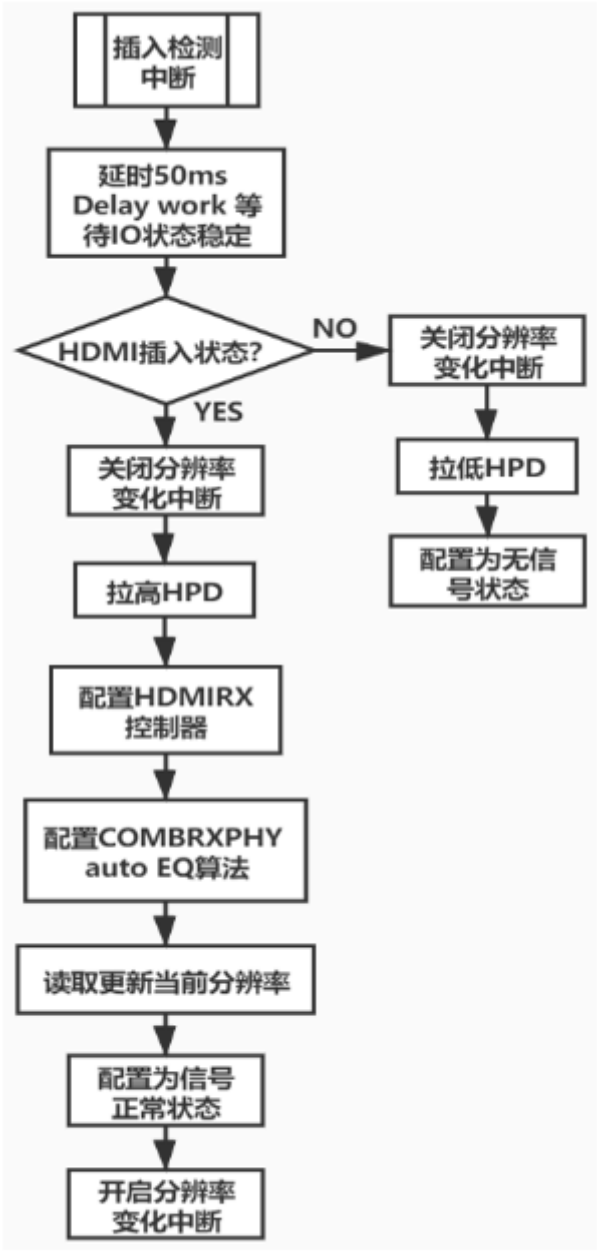
驱动主要流程参考RK628流程，主要包括：初始化、热拔插中断处理、分辨率切换中断处理。



驱动初始配置流程



分辨率变化中断处理程序



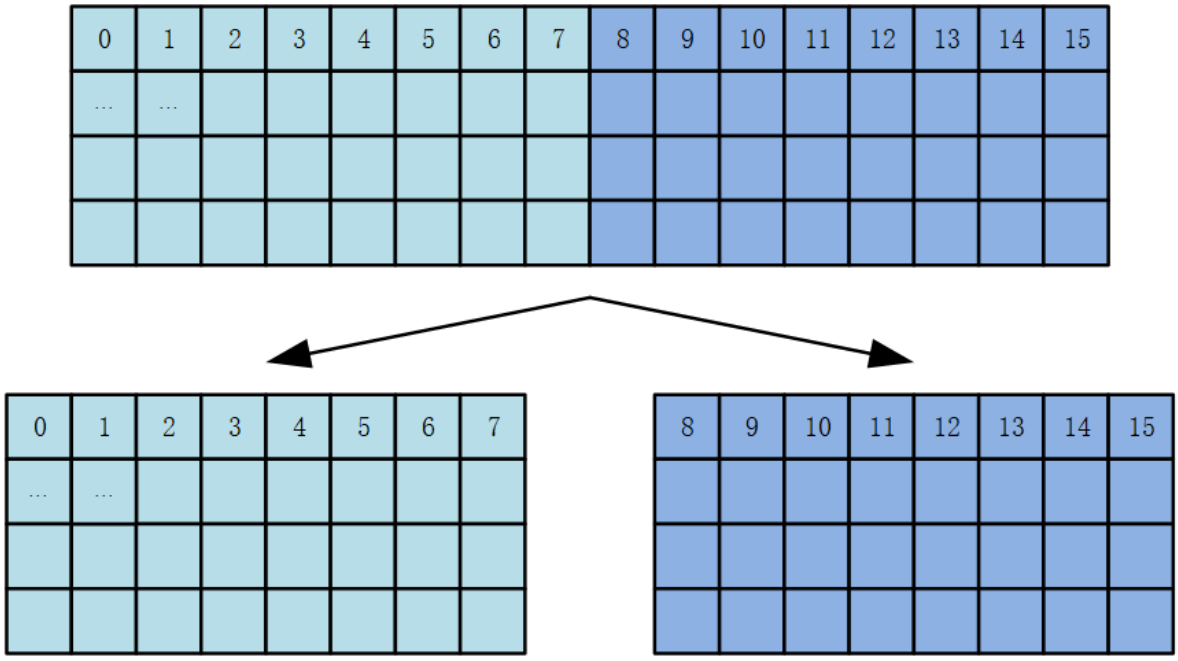
热拔插中断处理程序

2.3 双MIPI拼接方案

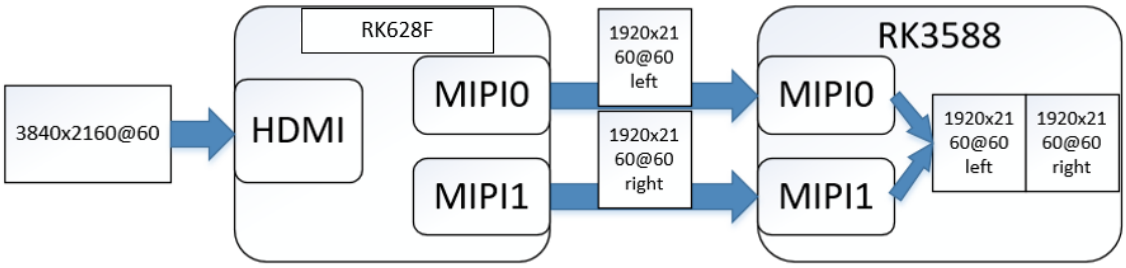
RK628F/H 受限于 MIPI DPHY 速率，一路 MIPI 最大只能支持 4K30，RK628F/H 有两路 MIPI-CSI，在对接 RK3588/RK3576 等时，4K60 的场景下，可以将 4K60 视频数据拆分成左右两半图像分别通过两路 4lane 的 MIPI-CSI/DSI 进行传输，由主控平台将图像进行合成为 4K60。

注：对接 RK3588/RK3576 SOC 平台，图像拼接合成直接在 VI 驱动完成，不会额外增加延时以及负载。

RK628F/H 内部支持可以将 HDMIRX 接收的图像 split 分割成左右两半，分别给到两路 MIPI-CSI/DSI 进行传输，如下图所示：



基于上述，在对接 RK3588/RK3576 有多路 MIPI 接口的 SOC 平台时，就可以采用双 MIPI 模式，接收下 4K60 分辨率的图像数据，功能流程如下图所示。



2.4 图像传输延时

- 对接TIF框架后加速显示，延时约为：30~50ms，叠加提前送帧低延时方案，延时约为15~30ms。
- 对接Camera框架，显示延时约为：100~120ms

3. 关键配置说明

3.1 RK628F/H DTS配置

转接芯片dts的配置如下，是i2c设备，需要配置到i2c总线下：

```
&i2c5 {
    status = "okay";
    pinctrl-0 = <&i2c5m3_xfer>;
    clock-frequency = <400000>;

    rk628_csi: rk628_csi@50 {
        reg = <0x50>;
```

```
        compatible = "rockchip,rk628-csi-v4l2";
        status = "okay";
        power-domains = <&power RK3576_PD_VI>;
        pinctrl-names = "default";
        pinctrl-0 = <&rk628_pin>;
        interrupt-parent = <&gpio3>;
        interrupts = <RK_PB0 IRQ_TYPE_LEVEL_HIGH>;
        enable-gpios = <&gpio3 RK_PD0 GPIO_ACTIVE_HIGH>;
        reset-gpios = <&gpio3 RK_PC5 GPIO_ACTIVE_HIGH>;
        plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
        continues-clk = <1>;
        cec-enable;
        #sound-dai-cells = <0>;

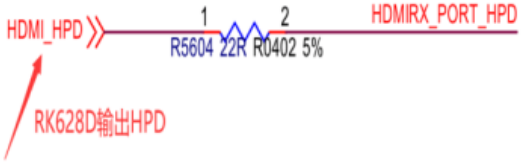
        rockchip,camera-module-index = <0>;
        rockchip,camera-module-facing = "back";
        rockchip,camera-module-name = "HDMI-MIPI1";
        rockchip,camera-module-lens-name = "RK628-CSI";

        multi-dev-info {
            dev-idx-l = <1>;
            dev-idx-r = <3>;
            combine-idx = <1>;
            pixel-offset = <0>;
            dev-num = <2>;
        };

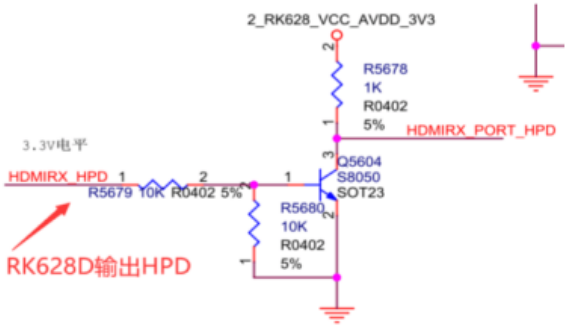
        port {
            hdmiin_out: endpoint {
                remote-endpoint = <&hdmi_mipi_in>;
                data-lanes = <1 2 3 4>;
            };
        };
    };
};
```

- interrupt-parent/ interrupts：连接RK628中断的GPIO引脚；
- enable-gpios：RK628供电控制GPIO引脚（若为常供电可不配置）；
- reset-gpios：RK628复位控制GPIO引脚
- hpd-output-inverted：HPD输出取反配置，若HPD输出电平在电路上做了取反，则需要使能此配置项；

◆ HPD未取反设计：

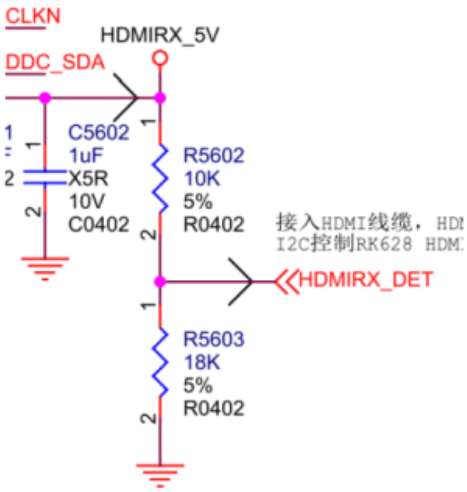


◆ HPD取反设计：

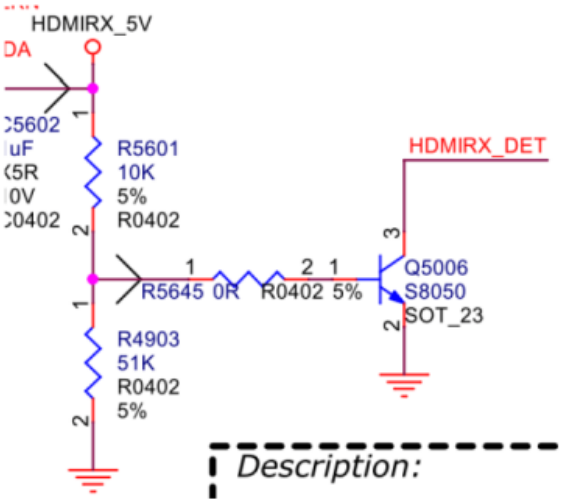


- plugin-det-gpios：HDMI插入检测GPIO引脚，注意电路上是否有对电平取反，有效电平需要配置正确；

◆ HDMIRX_DET未取反设计：



◆ HDMIRX_DET取反设计：



- rockchip,camera-module-name：对接TIF框架，该属性必须严格按照如下规律配置：HDMI-MIPIx，其中x为硬件实际接到哪个MIPI就配置哪个。RK3588按照如下规律配置。

> 若硬件接的是MIPI D/C PHY0 ， 则为HDMI-MIPI0或者HDMI-MIPI
>
> 若硬件接的是MIPI D/C PHY1 ， 则为HDMI-MIPI1
>
> 若硬件接的是MIPI CSI RX0 （4lane） ， 则为HDMI-MIPI2
>
> 若硬件接的是MIPI CSI RX0 （2lane-lane01） ， 则为HDMI-MIPI2
>
> 若硬件接的是MIPI CSI RX0 （2lane-lane23） ， 则为HDMI-MIPI3
>
> 若硬件接的是MIPI CSI RX1 （4lane） ， 则为HDMI-MIPI4
>
> 若硬件接的是MIPI CSI RX1 （2lane-lane01） ， 则为HDMI-MIPI4
>
> 若硬件接的是MIPI CSI RX1 （2lane-lane23） ， 则为HDMI-MIPI5

RK3576按照如下规律进行配置：

> 若硬件接的是MIPI D/C PHY，原理图上标注可能为CSI0 RX ， 则为HDMI-MIPI0或者HDMI-MIPI
>
> 若硬件接的是MIPI DPHY CSI1 RX （4lane） ， 则为HDMI-MIPI1
>
> 若硬件接的是MIPI DPHY CSI1 RX （2lane：lane01 + clk0） ， 则为HDMI-MIPI1
>
> 若硬件接的是MIPI DPHY CSI2 RX （2lane：lane23 + clk1） ， 则为HDMI-MIPI2
>
> 若硬件接的是MIPI DPHY CSI3 RX （4lane） ， 则为HDMI-MIPI3
>
> 若硬件接的是MIPI DPHY CSI3 RX（2lane：lane01） ， 则为HDMI-MIPI3
>
> 若硬件接的是MIPI DPHY CSI4 RX（2lane：lane23） ， 则为HDMI-MIPI4

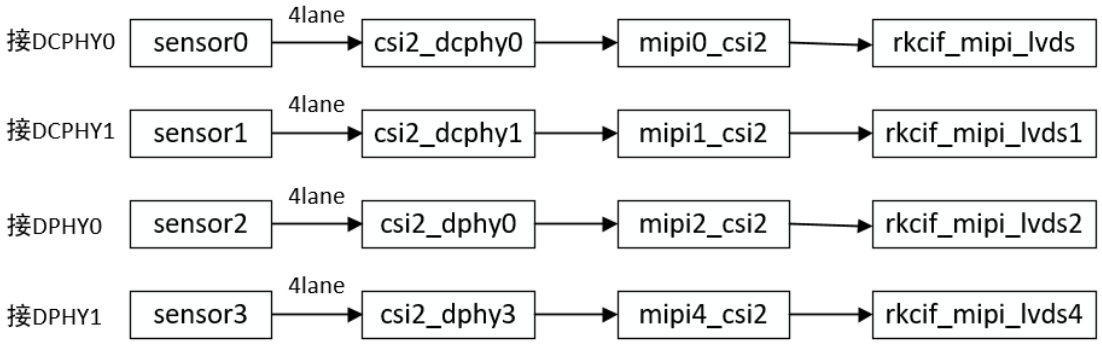
另外从DTS的pipeline链路也可以看出实际的配置，该属性与rkcif_mipi_lvds对应，rkcif_mipi_lvds对接的index是多少，则该属性也配置为多少。

3.2 Pipeline链路配置

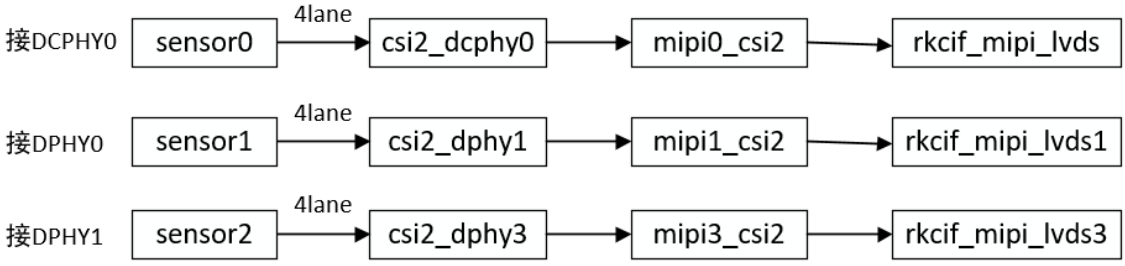
3.2.1 HDMI转MIPI链路

图像接收链路需要根据接的硬件接口进行配置，RK3588/RK3576有多路MIPI，需要根据实际链路进行配置。

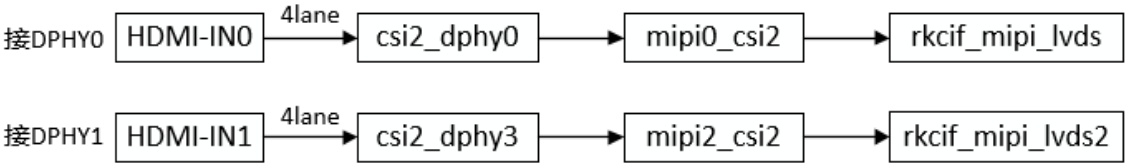
RK3588图像链路可参考如下：



RK3576参考如下：



RK3572的链路可以参考如下：



这里以RK628接RK3576的DPHY0为例：

```
&csi2_dphy0 {
    status = "okay";

    ports {
        #address-cells = <1>;
        #size-cells = <0>;
        port@0 {
            reg = <0>;
            #address-cells = <1>;
            #size-cells = <0>;

            hdmi_mipi_in: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&hdmiin_out>;
```

```
                data-lanes = <1 2 3 4>;
            };
        };
        port@1 {
            reg = <1>;
            #address-cells = <1>;
            #size-cells = <0>;

            csidphy0_out: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&mipi1_csi2_input>;
            };
        };
    };
};

&csi2_dphy0_hw {
    status = "okay";
};

&csi2_dphy1_hw {
    status = "okay";
};

&mipi1_csi2 {
    status = "okay";

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            #address-cells = <1>;
            #size-cells = <0>;

            mipi1_csi2_input: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&csidphy0_out>;
            };
        };

        port@1 {
            reg = <1>;
            #address-cells = <1>;
            #size-cells = <0>;

            mipi1_csi2_output: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&cif_mipi_in1>;
            };
        };
    };
};

&rkcif {
    status = "okay";
};
```

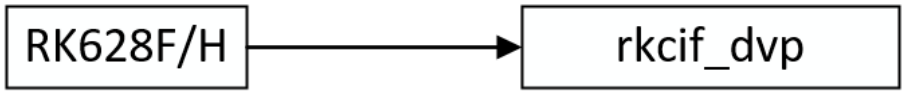
```
&rkcif_mipi_lvds1 {
    status = "okay";

    port {
        cif_mipi_in1: endpoint {
            remote-endpoint = <&mipi1_csi2_output>;
        };
    };
};

&rkcif_mmu {
    status = "okay";
};
```

3.2.2 HDMI转BT1120链路

RK628F/H支持HDMI转BT1120，并口输入到RK3588/RK3576，最大支持1080P60分辨率，图像数据流链路可参考如下：



相较于MIPI接口，链路配置比较简单，compatible需要修改为：rk628-bt1120-v4l2，需要更注意pinctrl对iomux的复用是否准确。dts配置参考如下：

```
&i2c4 {
    pinctrl-names = "default";
    pinctrl-0 = <&i2c4m1_xfer>;
    clock-frequency = <400000>;
    status = "okay";

    rk628_bt1120: rk628_bt1120@50 {
        compatible = "rockchip,rk628-bt1120-v4l2";
        reg = <0x50>;
        status = "okay";
        power-domains = <&power RK3576_PD_VI>;
        pinctrl-names = "default";
        pinctrl-0 = <&cif_dvp_clk &cif_dvp_bus16 &cif_dvp_bus8 &rk628_pin>;
        interrupt-parent = <&gpio2>;
        interrupts = <15 IRQ_TYPE_LEVEL_HIGH>;
        enable-gpios = <&gpio2 RK_PC0 GPIO_ACTIVE_HIGH>;
        reset-gpios = <&gpio2 RK_PB0 GPIO_ACTIVE_LOW>;
        plugin-det-gpios = <&gpio1 RK_PA2 GPIO_ACTIVE_LOW>;
        rockchip,camera-module-index = <0>;
        rockchip,camera-module-facing = "back";
        rockchip,camera-module-name = "RK628-BT1120";
        rockchip,camera-module-lens-name = "NC";
        dual-edge = <1>;

        port {
```

```
            rk628_out: endpoint {
                remote-endpoint = <&cif_para_in>;
                bus-width = <16>;
                pclk-sample = <1>;
            };
        };
    };
};

&rkcif_dvp {
    status = "okay";

    port {
        /* Parallel bus endpoint */
        cif_para_in: endpoint {
            remote-endpoint = <&rk628_out>;
        };
    };
};

&rkcif {
    status = "okay";
};

&rkcif_mmu {
    status = "okay";
};
```

注：另外需要注意，HDMI转BT1120的通路只能在camera框架下预览，TV目前不支持该通路。

3.3 双MIPI配置

RK628F/H若采用双MIPI的方案，Pipeline链路的配置与单MIPI基本无差别，pipeline按照CSI0的进行配置即可，RK628F/H的设备节点需要增加multi-dev-info的配置，解析如下：

> dev-idx-l：左半边图像对应的 mipi 接口编号
>
> dev-idx-r：右半边图像对应的 mipi 接口编号
>
> combine-idx：将左右两张图想合并到其中的一个 idx，dts 的链路以这个 idx 为准
>
> pixel-offset：像素偏移，默认设置 0
>
> dev-num：多少个 mipi 设备进行拼接

注：如果没有4K60的需求，只使用一个MIPI PORT的话，则不需要配置multi-dev-info的参数。

3.4 CSI/DSI模式配置

RK628F/H支持HDMI转MIPI-CSI或者DSI，MIPI-CSI仅支持YUV422格式输出，MIPI-DSI支持RGB888格式输出，若需要RGB888格式，则需要修改为DSI模式，DTS配置如下：

```
compatible = "rockchip,rk628-dsi-v4l2";
```

注：CSI模式仅支持YUV422输出，DSI模式仅支持RGB888输出，RGB888一般用在大屏对图像画质要求高的场景，要求RGB格式直接送显示，减少画质丢失，可走TV框架加速送显。在Camera框架下，HAL会将格式转换为NV12，同样存在画质丢失。

3.5 scaler 配置

RK628F/H 支持无论HDMI什么分辨率输入都固定缩放成固定的分辨率输出，DTS打开scaler的配置，驱动默认是按照1080P60进行设置。

DTS配置：

```
 scaler-en = <1>;
```

驱动固定输出分辨率，若需要其他分辨率，可自行修改。

```
static struct v4l2_dv_timings dst_timing = {
    .type = V4L2_DV_BT_656_1120,
    .bt = {
        .interlaced = V4L2_DV_PROGRESSIVE,
        .width = 1920,
        .height = 1080,
        .hfrontporch = 88,
        .hsync = 44,
        .hbackporch = 148,
        .vfrontporch = 4,
        .vsync = 5,
        .vbackporch = 36,
        .pixelclock = 148500000,
    },
};
```

3.6 HDCP

dts需要把hdcp-enable打开：

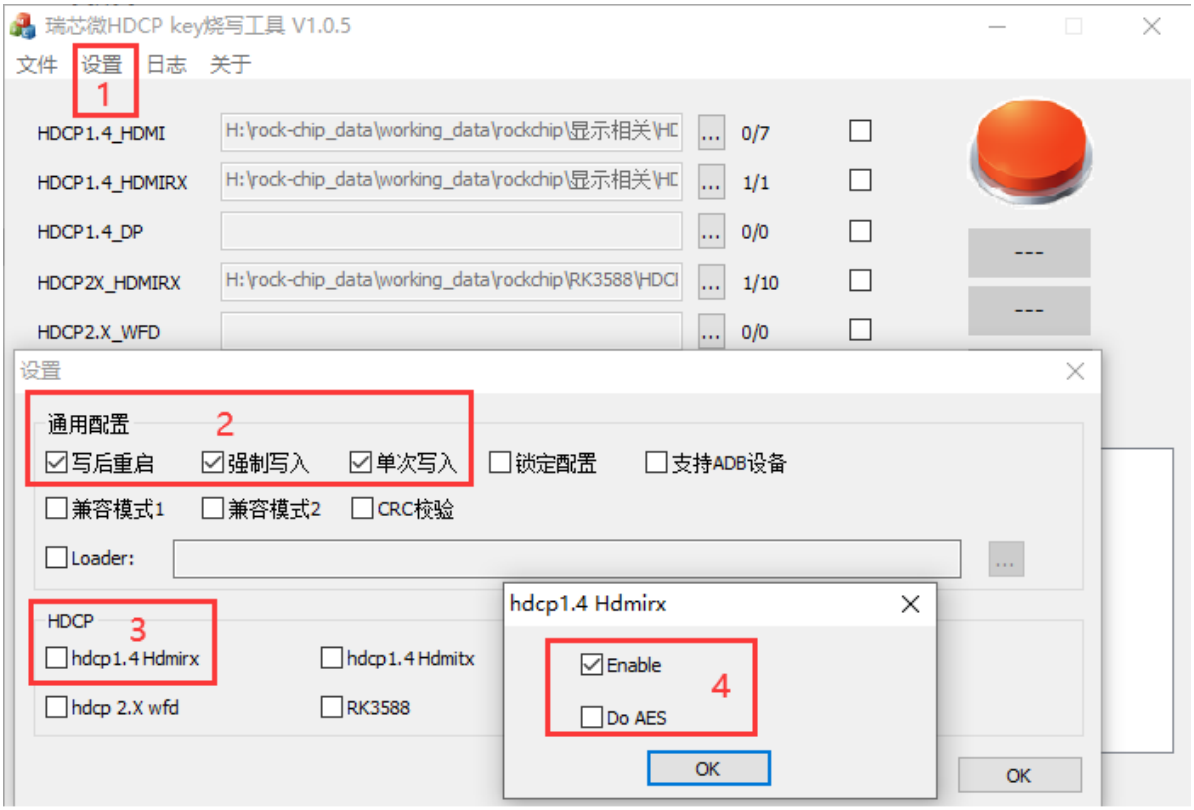
```
@@ -94,6 +94,7 @@ rk628_csi: rk628_csi@50 {
                plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
                continues-clk = <1>;
                cec-enable;
+               hdcp-enable;
                #sound-dai-cells = <0>;
```

dts配置上hdcp-enable之后，需要烧录HDCP1.4 RX KEY.

注：RK628只支持HDCP1.4.

烧写Key工具配置如下：

1. 点 `设置`
2. 勾选 `写后重启` `强制写入` `单次写入`
3. 勾选 `hdcp1.4 Hdmirx`
4. 勾选 `Eenable`，因为RK628不支持AES加密方式，所以不要选 `Do AES`



HDCP的状态可以通过节点`/d/rk628/5-0050/hdmirx/hdcp/status`查看。

3.7 CEC

1. dts需要把cec-enable打开：

```
@@ -94,6 +94,7 @@ rk628_csi: rk628_csi@50 {
                plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
                continues-clk = <1>;
+               cec-enable;
                #sound-dai-cells = <0>;
```

2. 需要把CEC服务打开，并配置为TV，device/rockchip/common修改如下:

```
diff --git a/rk3576_u/BoardConfig.mk b/rk3576_u/BoardConfig.mk
index eb52fb6..2482430 100755
--- a/rk3576_u/BoardConfig.mk
+++ b/rk3576_u/BoardConfig.mk
@@ -28,4 +28,8 @@ PRODUCT_KERNEL_CONFIG += pcie_wifi.config
 BOARD_GSENSOR_MXC6655XA_SUPPORT := true
 BOARD_CAMERA_SUPPORT_EXT := true
 BOARD_HS_ETHERNET := true
+BOARD_SUPPORT_HDMI_CEC := true
+BOARD_HDMI_CEC_TYPE := 0
```

```
BOARD_HDMI_CEC_TYPE的类型如下：
# 0         TV
# 1         Recording Device 1
# 2         Recording Device 2
# 3         Tuner 1
# 4         Playback Device 1
```

```
# 5         Audio System
# 6         Tuner 2
# 7         Tuner 3
# 8         Playback Device 2
# 9         Recording Device 3
# 10        Tuner 4
# 11        Playback Device 3
# 12        Reserved
# 13        Reserved
# 14        Specific Use
# 15        Unregistered (as Initiator address)
#           Broadcast (as Destination address)
```

注：`interrupts = <RK_PB0 IRQ_TYPE_LEVEL_HIGH>;` 中断的触发方式一定需要配置为电平触发，如果是边沿触发的话，会出现中断漏处理，CEC没有ACK导致通信异常。

3.8 低延时

3.8.1 VI提前送帧机制

如2.4所述，正常走TV框架可以将延时降低至30-50ms，在对延时更加敏感的场景，可以继续叠加VI低延时机制，将延时降低至20-30ms。设置如下节点可以开启低延时：

```
echo 100 > /sys/devices/platform/rkcif-mipi-lvds1/wait_line
echo 1 > /sys/devices/platform/rkcif-mipi-lvds1/low_latency
```

此处节点为rkcif-mipi-lvds1是因为DTS中配置到rkcif的节点是rkcif_mipi_lvds1。

3.8.2 使用限制

低延时机制在某些场景存在使用限制，目前存在如下限制：

- HDMI-IN帧率与显示帧率不匹配的场景下，低延时不生效
- RK3588平台，使能PQ之后，无法使用低延时机制
- RK3576打开PQ的DCI模块之后，低延时不生效
- 低延时只能使用在TV框架下，camera框架不支持低延时，camera框架下若打开低延时，可能会出现绿屏、黑屏、图像撕裂、图像错位、闪屏等异常现象。

3.9 色域转换说明

RK628F/H CSC增加了色域、格式转换矩阵，可以根据不同客户需求，支持灵活的色域转换，目前搭配RK3576根据客户需求，最新驱动代码提供一个接口，可设置range转换的输入range，输出range均默认设置为FULL，实际效果如下表格所示。

HDMI信号源range	用户设置参数	**RK628F CSC传入输入range**	**RK628F CSC传入输出range**	实际达成效果
Limit	Auto	Limit	Full	正常显示转换后的Full range效果
Limit	Limit	Limit	Full	正常显示转换后的Full range效果
Limit	Full	Full	Full	测试灰阶图，黑的不够黑，白的不够白 但不会并阶
Full	Auto	Full	Full	没有转换，正常显示
Full	Limit	Limit	Full	测试灰阶图，头尾的黑白并阶效果
Full	Full	Full	Full	没有转换，正常显示
Default	Auto	Full	Full	根据源的实际数据： 若信号源实际数据是Limit，则显示黑不够黑白不够白 若实际数据是Full，则正常显示
Default	Limit	Limit	Full	根据源的实际数据： 若信号源实际数据是Limit，则正常显示转换后的Full数据 若实际数据是Full，则头尾的黑白并阶效果
Default	Full	Full	Full	根据源的实际数据： 若信号源实际数据是Limit，则显示黑不够黑白不够白 若实际数据是Full，则正常显示

该设置通路，目前已经在TV框架上已经实现，打开PQ，APP可通过如下属性切换：

```
setprop persist.vendor.tvinput.rkpq.dst.range auto
setprop persist.vendor.tvinput.rkpq.dst.range limit
setprop persist.vendor.tvinput.rkpq.dst.range full
```

注：若上述range转换通路不满足客户需求，可将需求提供至redmine联系RK工程师。

3.10 EQ 配置说明

默认是自动 EQ，关闭CED(误码检测)，HDMIRX PHY 根据信号强度等自动计算 EQ，一般情况下没有问题，但是在信号质量差或是对于噪点敏感的情况下，可能会出现锁不住或是闪点的问题。对于不同的情况可以如下有不同的配置：

3.10.1 开启 CED

```
@@ -94,6 +94,7 @@ rk628_csi: rk628_csi@50 {
                plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
                continues-clk = <1>;
                cec-enable;
+               ced-enable;
                #sound-dai-cells = <0>;
```

在 CTS 测试的时候，或是需要检测误码的时候需要开启 CED，如上配置。

3.10.2 动态 EQ

```
@@ -94,6 +94,7 @@ rk628_csi: rk628_csi@50 {
                plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
                continues-clk = <1>;
                cec-enable;
+               dynamic-eq;
                #sound-dai-cells = <0>;
```

对于有些信号源锁不住或是分辨率识别出错，或是出现噪点的情况下，可以开启动态EQ，原理是先自动EQ，如果锁不住或是 CED 检测有误码，则从预设的 EQ 列表中选择对应的 EQ 尝试锁定并判断误码，然后选出最优的 EQ 值。dynamic-eq 配置上的话，会自动开启 ced-enable.

3.10.3 固定 EQ

```
@@ -94,6 +94,9 @@ rk628_csi: rk628_csi@50 {
                plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
                continues-clk = <1>;
                cec-enable;
+               force-eq14 = /bits/ 8 <0x1f 0x1f 0x1f>;
+               force-eq20 = /bits/ 8 <0x21 0x21 0x21>;
                #sound-dai-cells = <0>;
```

force-eq14 表示 HDMI1.4 对应 3 个 channel 的 EQ 值；

force-eq20 表示 HDMI2.0 对应 3 个 channel 的 EQ 值。

一般在前级有 Switch 的情况下配置，因为前面 Switch的话，信号重构过，相对比较稳定，可以使用固定 EQ.

3.11 HDR10

限制：仅 kernel-6.1，并且走 TV 框架。

支持 HDR10 的话，EDID 与一般情况下会有不同，所以需要配置 hdr-support:

```
@@ -94,6 +94,7 @@ rk628_csi: rk628_csi@50 {
                plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
                continues-clk = <1>;
                cec-enable;
+               hdr-support;
                #sound-dai-cells = <0>;
```

如果需要 TV 与 Camera 相互切换的情况下，可以选择不支持 HDR 或是 应用需要自己切对应的 EDID.

可以通过下面的指令查看 HDR 相关信息：

```
# cat /d/rk628/3-0050/hdmirx/status
status: plugin
Clk-Ch:Lock     Ch0:Lock        Ch1:Lock        Ch2:Lock
Color Format: YUV422
Timing: 3840x2160p60 (4400x2250)                hfp:176  hs:88  hbp:296  vfp:8
vs:10  vbp:72
Pixel Clk: 594000000
Mode: HDMI
Color Depth: 8 bit
Color Range: Full
Color Space: BT2020_RGB_OR_YCbCr
EOTF: ST2084
x0: 12800                               y0: 12800
x1: 12800                               y1: 12800
x2: 12800                               y2: 12800
white x: 12800                  white y: 256
max lum: 256                    min lum: 256
max cll: 256                    max fall: 0
```

这边 Color Depth: 8 bit 是因为 HDMIRX YUV422 都按 12 bit 接收，没有做区分，只有在识别到 EOTF: ST2084 的情况下，以10 bit 输出。

在显示设备支持 HDR 的情况下是 HDR2HDR，效果以显示器的处理结果为准。

在显示设备不支持 HDR 的情况下是 HDR2SDR，效果因 VOP 的配置参数不同会有差异。

支持HDR，内核代码需要包括如下支持（目前对接RK3588/RK3576支持HDR）：

media/i2c/rk628：rk628驱动代码需要更新到最新。

media/platform/rockchip/cif/：需要包括如下提交：

```
media: platform: rockchip: vicap yuv422-10bit

Signed-off-by: Jianwei Fan <jianwei.fan@rock-chips.com>
Change-Id: I4d7321a137ee5e516c3d1c59abe5fabab2d50156
```

注：如果需要支持HDR，但是代码不符合，请redmine联系RK获取。

4. HDMIIN APK 适配方法

4.1 APK源码

- packages/apps/TV/partner_support/samples ：提供TV源数据服务，通过framework与HAL层、预览APK进行交互，由于是开机运行的隐藏服务，该APK在桌面上是隐藏图标的。
- hardware/rockchip/tv_input ：TVHAL层代码，开关流、热拔插和分辨率切换事件等与驱动进行命令交互。
- hardware/rockchip/camera：cameraHAL层代码，camera框架取流、拍照等功能实现。
- vendor/rockchip/hardware/interfaces/hdmi：走camera框架使用，负责监听分辨率变化与热拔插事件，与驱动以及APK交互。
- packages/apps/rkCamera2 ：预览apk，此应用有2个界面。默认的MainActivity界面使用TIF方案进行预览，通过framework层与上述TV源数据服务进行交互；RockchipCamera2界面使用Camera方案进行预览，用标准的Camera API，对MIPI的节点的cameraid进行open操作。
  APK在桌面上图标名称为 HdmiIn ，通常客户会二次开发替换为自己的预览APK。

4.2 TV框架适配

- SDK默认代码HDMI IN功能是关闭的，使能HDMI IN功能，需配置如下属性，开启后会编译含上述APK在内的相关模块：

```
vim device/rockchip/rk3588/BoardConfig.mk
BOARD_HDMI_IN_SUPPORT := true
```

APK支持RK3588 HDMI RX通路数据预览和HDMI转MIPI-CSI通路数据预览，使用时需要切换。

TIF预览方式，需要设置MIPI-CSI2通路：

```
setprop tvinput.hdmiin.type 1
```

注意：如果产品没有HDMI RX，只有RK628F/H，例如RK3576，RK3588S以及RK3588 dts禁用HDMI RX的场景，需要把tvinput.hdmiin.type属性配置到系统中，否则TV方案会黑屏无法预览RK628F/H，RK3576的举例配置如下：

```
device/rockchip/rk3576/device.mk
PRODUCT_PROPERTY_OVERRIDES += \
    tvinput.hdmiin.type=1
```

4.3 Camera框架适配

4.3.1 打开宏配置

- 走camera框架需要打开：

  ```
  vim device/rockchip/rk3588/BoardConfig.mk
  CAMERA_SUPPORT_HDMI := true
  ```

只配置CAMERA_SUPPORT_HDMI，而不配置BOARD_HDMI_IN_SUPPORT的情况下。如想要用rkCamera2进行camera预览，需要将rkCamera2加入到编译并配置属性persist.sys.hdmiinmode值为2。其他补充见文档：

《Rockchip_Developer_Guide_HDMI_RX_CN》

```
PRODUCT_PACKAGES += \
    rkCamera2
```

4.3.2 camera3_profiles.xml配置

走camera框架预览需要适配camera3_profiles.xml，注册camera设备。

camera3_profiles.xml文件对应SDK目录下具体芯片平台的文件：

```
hardware/rockchip/camera/etc/camera/camera3_profiles_rk3xxx.xml
```

主要配置注意事项如下，详情可参考SOC Sensor的配置方法：

- **name**：需要与驱动名称一致，有大小写区别；
- **moduleId**：需要与驱动dts中配置的index一致；

```
    </Profiles>
    <Profiles cameraId="0" name="rk628-csi" moduleId="m00">
        <Supported_hardware>
            <hwType value="SUPPORTED_HW_RKISP1"/>
```

```
&rk628_csi {
        status = "okay";
        /*
         * If the hpd output level is inverted on the circuit,
         * the following configuration needs to be enabled.
         */
        /* hpd-output-inverted; */
        plugin-det-gpios = <&gpio0 13 GPIO_ACTIVE_HIGH>;
        power-gpios = <&gpio0 17 GPIO_ACTIVE_HIGH>;
        rockchip,camera-module-index = <0>;
        rockchip,camera-module-facing = "back";
        rockchip,camera-module-name = "RK628-CSI";
        rockchip,camera-module-lens-name = "NC";
```

- scaler.availableStreamConfigurations/scaler.availableMinFrameDurations/ scaler.availableStallDurations：需要正确配置驱动支持的分辨率和最小的帧间隔时间，若驱动中需要增加新的分辨率支持，在这里也要相应地增加配置；

```
<scaler.availableStreamConfigurations value="BLOB,3840x2160,OUTPUT,
    BLOB,1920x1080,OUTPUT,
    BLOB,1280x720,OUTPUT,
    BLOB,720x576,OUTPUT,
    BLOB,720x480,OUTPUT,
    YCbCr_420_888,3840x2160,OUTPUT,
    YCbCr_420_888,1920x1080,OUTPUT,
    YCbCr_420_888,1280x720,OUTPUT,
    YCbCr_420_888,720x576,OUTPUT,
    YCbCr_420_888,720x480,OUTPUT,
    IMPLEMENTATION_DEFINED,3840x2160,OUTPUT,
    IMPLEMENTATION_DEFINED,1920x1080,OUTPUT,
    IMPLEMENTATION_DEFINED,1280x720,OUTPUT,
    IMPLEMENTATION_DEFINED,720x576,OUTPUT,
    IMPLEMENTATION_DEFINED,720x480,OUTPUT" />
<scaler.availableMinFrameDurations value="BLOB,3840x2160,33333333,
    BLOB,1920x1080,16666667,
    BLOB,1280x720,16666667,
    BLOB,720x576,20000000,
    BLOB,720x480,16666667,
    YCbCr_420_888,3840x2160,33333333,
    YCbCr_420_888,1920x1080,16666667,
    YCbCr_420_888,1280x720,16666667,
    YCbCr_420_888,720x576,20000000,
    YCbCr_420_888,720x480,16666667,
    IMPLEMENTATION_DEFINED,3840x2160,33333333,
    IMPLEMENTATION_DEFINED,1920x1080,16666667,
    IMPLEMENTATION_DEFINED,1280x720,16666667,
    IMPLEMENTATION_DEFINED,720x576,20000000,
    IMPLEMENTATION_DEFINED,720x480,16666667"/>
<scaler.availableStallDurations value="BLOB,3840x2160,33333333,
    BLOB,1920x1080,16666667,
    BLOB,1280x720,16666667,
    BLOB,720x576,20000000,
    BLOB,720x480,16666667"/>
```

- **sensor.orientation**：图像旋转角度，支持0、90、180、270；

```
<sensor.maxAnalogSensitivity value="2400"/> <!-- HAL
<sensor.orientation value="0"/>
<sensor.profileHueSatMapDimensions value="0,0,0"/>
```

4.3.3 切换预览

camera预览方式，需要打开RockchipCamera2界面，注意编译的时候需要使能CAMERA_SUPPORT_HDMI。

```
setprop persist.sys.hdmi.port mipi
setprop persist.sys.hdmiinmode 2
```

persist.sys.hdmi.port是camera框架下，mipi与hdmirx的互相切换，rk628是走HDMI转MIPI的方式，故设置mipi，如果是没有hdmirx的场景，如RK3576等，该属性也可以缺省。

persist.sys.hdmiinmode是tv框架与camera框架切换，camera设置为2。

4.4 TIF预览与camera预览差异

	TIF	Camera
优点	延迟低	app端能够拿到预览数据进行后处理
缺点	不支持屏幕旋转、分屏、画中画功能； app端拿不到预览buffer数据； 不支持screencap方式的截图命令	延迟高于TIF

1. TIF预览不支持画中画功能，要使用画中画，可以从TIF切换为camera方案。在TIF预览示例MainActivity中，点击屏幕任一位置，弹出框的“PIP”按钮，提供了从TIF预览切换到camera画中画预览的功能，要查看该效果，前提需确保CAMERA_SUPPORT_HDMI := true，即camera预览方案是使能状态。
2. TIF预览下，支持谷歌标准的MediaProjectionManager创建虚拟屏方式进行录像与截图，也支持screenrecord命令录屏。

   如果有上述录屏和截图需求，或者需要pc通过adb投屏进行投屏操作，需要配置属性：

   ```
   vim device/rockchip/rk3588/device.mk
   PRODUCT_PROPERTY_OVERRIDES += debug.sf.enable_hwc_vds=true
   ```

   如果有重载过 device 下的产品目录，需将其配置在对应产品的目录下，可在开机后通过 adb 执行 getprop debug.sf.enable_hwc_vds 查看打印值是否为 true确认是否改动有效。
3. TIF预览下，不支持adb的screencap命令截图，包括音量键+电源键这类快捷按钮截图，需要上述第2点提到的谷歌标准MediaProjectionManager虚拟屏截图。

5. RK628F/H 使用一些限制说明

5.1 特殊Timing限制

RK628F/H 没有频点限制，但是对某些HDMIRX的特殊时序无法支持，或者需要特殊处理。

5.1.1 隔行分辨率

RK628F/H 不支持隔行分辨率。

5.1.2 最大分辨率限制

RK628F/H HDMI 输入支持的最大分辨率为4096*2160P60，594M。

5.1.3 HBP限制

RK628F/H HBP有限制，HBP 的 寄存器只有9 bit，最大支持到 511，RK628F/H不支持HBP大于511的timing，如果遇到该类信号源，建议通过EDID来限制（默认EDID不带这类timing，常见于使用信号发生器测试场景）。

5.1.4 VFP限制

对于部分VFP小于等于1的Timing，可能会出现画面异常现象，如：左下角出现画面花屏。

出现该现象的原因是：VFP太小，RK628F/H的vst 最小只能设置为1，相当于只有1行的缓存，但是实际前后级的时钟存在偏差，两边时钟一帧的累计偏差超过一行的话，就会出现画面花屏的异常。

出现此类问题推荐使用EDID限制的方式解决。

5.1.5 Scaler限制

RK628F/H 支持分辨率固定输出，内部会Scaler，但不能改变帧率，只能对图像进行缩放。

Scaler功能需要包括如下提交，可以解决部分Timing在Scaler的时候可能会出现花屏异常的问题。

```
commit 215d3c5bf9aa24c769ef5dbfaa937c75a4b5c59e
Author: Jianwei Fan <jianwei.fan@rock-chips.com>
Date:   Wed Apr 30 15:45:16 2025 +0800

    media: i2c: rk628: fix vsync/vfp/vbp calculate when scaler en

    Signed-off-by: Jianwei Fan <jianwei.fan@rock-chips.com>
    Change-Id: I4472e8ceff09e326107c203ae6d65eaf907e90ac
```

5.2 CSI/DSI格式说明

输出接口	输出图像格式	最大分辨率	最大MIPI速率	其他说明
CSI	YUV422	双MIPI 4K60 单MIPI 4K30	1.3Gbps/lane	
CSI	YUV422-10bit	双MIPI 4K60 单MIPI 4K30	1.5Gbps/lane	
DSI	RGB888	双MIPI 4K60 单MIPI 1080P60	1.8Gbps/lane	4K60 1.8G场景需要1.2V供电

5.3 对接RK3588 D/CPHY说明

RK3588 D/CPHY 对TX端的MIPI 结束Lp11的时序有严格的限制要求，RK628F/H MIPI -CSI接口不满足时序的要求，但是RK628F/H的DSI接口，可以满足时序的要求，因此对接RK3588 DCPHY的场景，建议使用DSI接口，RGB格式，使用TV框架直接预览。

DCPHY的参数配置如下：

```
static struct rkmodule_csi_dphy_param rk3588_dcphy_param = {
    .vendor = PHY_VENDOR_SAMSUNG,
    .lp_vol_ref = 0,
    .lp_hys_sw = {3, 0, 0, 0},
    .lp_escclk_pol_sel = {1, 0, 0, 0},
    .skew_data_cal_clk = {0, 3, 3, 0},
    .clk_hs_term_sel = 2,
    .data_hs_term_sel = {2, 2, 2, 2},
    .reserved = {0},
};
```

6. 常用调试方法

6.1 打开log开关

一般调试过程中建议打开debug的log开关：

打开RK628F/H 驱动debug开关，其中x为挂载的i2c，50为RK628F 的i2c地址

```
echo 2 > /sys/module/rk628_csi/parameters/debug
echo 1 > /d/rk628/x-0050/debug
dmesg -n 8
```

打开vicap驱动debug开关：

```
echo 3 > /sys/module/video_rkcif/parameters/debug
echo 8 > /proc/sys/kernel/printk
```

打开cameraHAL3 debug开关：

```
setprop persist.vendor.camera.hal.debug 5
```

打开TV debug开关

```
setprop vendor.tvinput.debug.level 3
```

6.2 查询驱动timings信息

```
rk3588_s:/ # v4l2-ctl -d /dev/v4l-subdev2 --query-dv-timings
    Active width: 1920
    Active height: 1080
    Total width: 2200
    Total height: 1125
    Frame format: progressive
    Polarities: -vsync -hsync
    Pixelclock: 0 Hz (0.00 frames per second)
    Horizontal frontporch: 88
    Horizontal sync: 0
    Horizontal backporch: 192
    Vertical frontporch: 9
    Vertical sync: 0
    Vertical backporch: 36
    Standards:
```

6.3 查看media拓扑结构

HDMI2MIPI驱动框架类似camera，需要保证pipeline的完整才可以正常工作，使用media-ctl查看pipeline：

```
media-ctl -d /dev/mediaX -p      //X=0123...
```

以RK3588 + RK628为例，正常结果pipeline显示如下：

```
|rk3588_s:/ # media-ctl -p
Opening media device /dev/media0
Enumerating entities
Found 14 entities
Enumerating pads and links
Media controller API version 0.0.160

Media device information
------------------------
driver          rkcif
model           rkcif-mipi-lvds
serial
bus info
hw revision     0x0
driver version  0.0.160

Device topology
- entity 1: stream_cif_mipi_id0 (1 pad, 11 links)
             type Node subtype V4L
             device node name /dev/video0
        pad0: Sink
                 <- "rockchip-mipi-csi2":1 [ENABLED]
                 <- "rockchip-mipi-csi2":2 []
                 <- "rockchip-mipi-csi2":3 []
                 <- "rockchip-mipi-csi2":4 []
                 <- "rockchip-mipi-csi2":5 []
                 <- "rockchip-mipi-csi2":6 []
```

```
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 []
                <- "rockchip-mipi-csi2":11 []

- entity 5: stream_cif_mipi_id1 (1 pad, 11 links)
            type Node subtype V4L
            device node name /dev/video1
        pad0: Sink
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 [ENABLED]
                <- "rockchip-mipi-csi2":3 []
                <- "rockchip-mipi-csi2":4 []
                <- "rockchip-mipi-csi2":5 []
                <- "rockchip-mipi-csi2":6 []
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 []
                <- "rockchip-mipi-csi2":11 []

- entity 9: stream_cif_mipi_id2 (1 pad, 11 links)
            type Node subtype V4L
            device node name /dev/video2
        pad0: Sink
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 []
                <- "rockchip-mipi-csi2":3 [ENABLED]
                <- "rockchip-mipi-csi2":4 []
                <- "rockchip-mipi-csi2":5 []
                <- "rockchip-mipi-csi2":6 []
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 []
                <- "rockchip-mipi-csi2":11 []

- entity 13: stream_cif_mipi_id3 (1 pad, 11 links)
             type Node subtype V4L
             device node name /dev/video3
        pad0: Sink
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 []
                <- "rockchip-mipi-csi2":3 []
                <- "rockchip-mipi-csi2":4 [ENABLED]
                <- "rockchip-mipi-csi2":5 []
                <- "rockchip-mipi-csi2":6 []
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 []
                <- "rockchip-mipi-csi2":11 []

- entity 17: rkcif_scale_ch0 (1 pad, 11 links)
             type Node subtype V4L
             device node name /dev/video4
        pad0: Sink
```

```
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 []
                <- "rockchip-mipi-csi2":3 []
                <- "rockchip-mipi-csi2":4 []
                <- "rockchip-mipi-csi2":5 [ENABLED]
                <- "rockchip-mipi-csi2":6 []
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 []
                <- "rockchip-mipi-csi2":11 []

- entity 21: rkcif_scale_ch1 (1 pad, 11 links)
             type Node subtype V4L
             device node name /dev/video5
        pad0: Sink
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 []
                <- "rockchip-mipi-csi2":3 []
                <- "rockchip-mipi-csi2":4 []
                <- "rockchip-mipi-csi2":5 []
                <- "rockchip-mipi-csi2":6 [ENABLED]
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 []
                <- "rockchip-mipi-csi2":11 []

- entity 25: rkcif_scale_ch2 (1 pad, 11 links)
             type Node subtype V4L
             device node name /dev/video6
        pad0: Sink
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 []
                <- "rockchip-mipi-csi2":3 []
                <- "rockchip-mipi-csi2":4 []
                <- "rockchip-mipi-csi2":5 []
                <- "rockchip-mipi-csi2":6 []
                <- "rockchip-mipi-csi2":7 [ENABLED]
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 []
                <- "rockchip-mipi-csi2":11 []

- entity 29: rkcif_scale_ch3 (1 pad, 11 links)
             type Node subtype V4L
             device node name /dev/video7
        pad0: Sink
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 []
                <- "rockchip-mipi-csi2":3 []
                <- "rockchip-mipi-csi2":4 []
                <- "rockchip-mipi-csi2":5 []
                <- "rockchip-mipi-csi2":6 []
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 [ENABLED]
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 []
```

```
                <- "rockchip-mipi-csi2":11 []

- entity 33: rkcif_tools_id0 (1 pad, 11 links)
              type Node subtype V4L
              device node name /dev/video8
        pad0: Sink
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 []
                <- "rockchip-mipi-csi2":3 []
                <- "rockchip-mipi-csi2":4 []
                <- "rockchip-mipi-csi2":5 []
                <- "rockchip-mipi-csi2":6 []
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 [ENABLED]
                <- "rockchip-mipi-csi2":10 []
                <- "rockchip-mipi-csi2":11 []

- entity 37: rkcif_tools_id1 (1 pad, 11 links)
              type Node subtype V4L
              device node name /dev/video9
        pad0: Sink
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 []
                <- "rockchip-mipi-csi2":3 []
                <- "rockchip-mipi-csi2":4 []
                <- "rockchip-mipi-csi2":5 []
                <- "rockchip-mipi-csi2":6 []
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 [ENABLED]
                <- "rockchip-mipi-csi2":11 []

- entity 41: rkcif_tools_id2 (1 pad, 11 links)
              type Node subtype V4L
              device node name /dev/video10
        pad0: Sink
                <- "rockchip-mipi-csi2":1 []
                <- "rockchip-mipi-csi2":2 []
                <- "rockchip-mipi-csi2":3 []
                <- "rockchip-mipi-csi2":4 []
                <- "rockchip-mipi-csi2":5 []
                <- "rockchip-mipi-csi2":6 []
                <- "rockchip-mipi-csi2":7 []
                <- "rockchip-mipi-csi2":8 []
                <- "rockchip-mipi-csi2":9 []
                <- "rockchip-mipi-csi2":10 []
                <- "rockchip-mipi-csi2":11 [ENABLED]

- entity 45: rockchip-mipi-csi2 (12 pads, 122 links)
              type V4L2 subdev subtype Unknown
              device node name /dev/v4l-subdev0
        pad0: Sink
                [fmt:UYVY2X8/1920x1080
                 crop.bounds:(0,0)/1920x1080
                 crop:(0,0)/1920x1080]
                <- "rockchip-csi2-dphy0":1 [ENABLED]
```

```
        pad1: Source
                -> "stream_cif_mipi_id0":0 [ENABLED]
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 []
        pad2: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 [ENABLED]
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 []
        pad3: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 [ENABLED]
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 []
        pad4: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 [ENABLED]
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 []
        pad5: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 [ENABLED]
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
```

```
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 []
        pad6: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 [ENABLED]
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 []
        pad7: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 [ENABLED]
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 []
        pad8: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 [ENABLED]
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 []
        pad9: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 [ENABLED]
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 []
        pad10: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
```

```
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 [ENABLED]
                -> "rkcif_tools_id2":0 []
        pad11: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 [ENABLED]

- entity 58: rockchip-csi2-dphy0 (2 pads, 2 links)
             type V4L2 subdev subtype Unknown
             device node name /dev/v4l-subdev1
        pad0: Sink
                [fmt:UYVY2X8/1920x1080]
                <- "m00_b_rk628-csi 5-0050":0 [ENABLED]
        pad1: Source
                -> "rockchip-mipi-csi2":0 [ENABLED]

- entity 63: m00_b_rk628-csi 5-0050 (1 pad, 1 link)
             type V4L2 subdev subtype Sensor
             device node name /dev/v4l-subdev2
        pad0: Source
                [fmt:UYVY2X8/1920x1080]
                -> "rockchip-csi2-dphy0":0 [ENABLED]
```

6.4 查看RK628F/H HDMI-RX状态

可通过如下指令读取RK628F HDMI-RX的信息，包括分辨率信息、锁定状态、拔插状态、颜色格式等等：

```
console:/ # cat /sys/kernel/debug/rk628/3-0050/hdmirx/status
status: plugin
Clk-Ch:Lock     Ch0:Lock        Ch1:Lock        Ch2:Lock
Color Format: RGB
Timing: 1920x1080p60 (2200x1125)                hfp:88  hs:45  hbp:147  vfp:4
vs:5  vbp:36
Pixel Clk: 148500000
Mode: HDMI
Color Depth: 8 bit
Color Range: Limited
Color Space: RGB
```

6.5 RK628F/H 设置pattern输出

RK628F 支持设置scaler color bar输出，具体使用命令如下：

```
echo 1 > /sys/kernel/debug/rk628/3-0050/scaler_color_bar

//pattern 模式选择
echo 2 > /sys/kernel/debug/rk628/3-0050/scaler_color_bar
```

6.6 RK628F/H 寄存器读写

RK628 寄存器调试节点如下，当前示例 RK628 接在 I2C3，地址为 0x50：

```
console:/ # ls /sys/kernel/debug/rk628/3-0050/registers/
adapter    combtxphy  csi    dsi0  gpio0  gpio2  grf      hdmirxphy
combrxphy  cru        csi1   dsi1  gpio1  gpio3  hdmirx
```

寄存器节点可支持读写。

读寄存器：

```
console:/ # cat /sys/kernel/debug/rk628/3-0050/registers/cru
rk628_cru:

0xc0000: 00001031 00000441 00800000 00000007
0xc0010: 00007f00
0xc0020: 00001028 00000441 00f5c28f 00000007
0xc0030: 00007f00
0xc0040: 00005019 00000541 0099735e 00000007
0xc0050: 00007f00
0xc0060: 00000015
0xc0080: 00008989 00000003 00000007 00004b80
0xc0090: 03355460 000080d7 00008004 0000003b
0xc00a0: 00002c00 00000000 00000000 00000000
0xc00b0: 00285f58 00010008 00010008 00000103
0xc00c0: 00000300 00000703 0000000b 00000008
0xc00d0: 0001000a 00000b07
0xc0180: 00000000 00000000 00000000 00000000
0xc0190: 00000000 00000000
0xc0200: 00000000 00000000 00000000 00000000
0xc0210: 00000000
```

写寄存器：

```
rk3288:/ # echo 0x000 0xffffffff > /sys/kernel/debug/rk628/3-0050/registers/grf
```

6.7 HDCP调试指令

使能HDCP：

```
echo 1 > /d/rk628/5-0050/hdmirx/hdcp/enable
```

查看HDCP的状态

```
cat /d/rk628/5-0050/hdmirx/hdcp/status
```

6.8 开启图像数据流

开启图像数据流命令如下所示，需要根据实际情况，设置video节点、分辨率、格式等信息，video节点可以按照如下方法确认：使用上述查看media拓扑命令找到转接芯片所在的media节点并查看对应的pipeline，pipeline中节点名为：stream_cif_mipi_id0所对应的video节点既是取流的节点：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=3840,height=2160,pixelformat='NV12' --stream-mmap=4
```

如果是DSI RGB888格式，取流命令如下：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=3840,height=2160,pixelformat='RGB3' --stream-mmap=4
```

如果是HDR场景，主控输出的是YUYV 10bit的格式，取流命令如下：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=3840,height=2160,pixelformat='Y210' --stream-mmap=4
```

6.9 抓取图像文件

可以将图像保存到文件，通过adb pull 到设备，使用7yuv，YUVview等工具软件查看：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=3840,height=2160,pixelformat='NV12' --stream-mmap=3 --stream-skip=4 -
-stream-to=/data/3840x2160_nv12.yuv --stream-count=5 --stream-poll
```

如果是DSI RGB888格式，抓取图像命令如下：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=3840,height=2160,pixelformat='RGB3' --stream-mmap=3 --stream-skip=4 -
-stream-to=/data/3840x2160_rgb.yuv --stream-count=5 --stream-poll
```

如果是HDR场景，抓取图像命令如下：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=3840,height=2160,pixelformat='Y210' --stream-mmap=3 --stream-skip=4 -
-stream-to=/data/3840x2160_rgb.yuv --stream-count=5 --stream-poll
```

6.10 TV预览抓图

TV若在预览是出现图像异常，可以通过TV dump图像来判断是前级驱动的问题还是后级显示的问题，TV预览场景下抓图如下步骤：

```
su
setenforce 0
rm -rf data/system/dumpimage
mkdir data/system/dumpimage
chmod 777 data/system/dumpimage
setprop vendor.hdmiin.debug.dump 1
setprop vendor.hdmiin.debug.dumpnum 10
setprop vendor.tvinput.debug.dump 1
setprop vendor.tvinput.debug.dumpnum 10
0.5s之后执行：
setprop vendor.hdmiin.debug.dumpnum 0
setprop vendor.tvinput.debug.dumpnum 0
2s后执行：
setprop vendor.hdmiin.debug.dump 0
setprop vendor.tvinput.debug.dump 0
```

执行完成后将data/system/dumpimage 文件从设备取出，使用7YUV工具进行查看。

6.11 查看camera是否注册成功

如下命令可以查看camera是否注册成功，如果没注册上，请检查camera3_profiles.xml文件。

```
dumpsys media.camera
```

6.12 camera预览方式抓图

camera方式预览的话可以按照如下方式抓图，如下设置抓图会抓取预览的图像以及从video取得的原始图像，方便判断是驱动问题还是HAL处理图像数据问题。

```
adb root
adb remount
adb shell setprop persist.vendor.camera.dump 139
adb shell setprop persist.vendor.camera.dump.cnt 20 设置dump20帧
adb shell setenforce 0
adb shell mkdir /data/dump
adb shell chmod 777 /data/dump/
adb shell setprop  persist.vendor.camera.dump.path /data/dump/

打开预览，导出/data/dump/
```

6.13 抓取图层信息

TV显示异常的时，可以通过如下方式抓取图层信息

```
cat /d/dri/0/summary
```

7. 常见问题

取数据流等调试常见问题可参考文档：
《Rockchip_Developer_Guide_Android12+_HDMI_IN_Bridge_CN》

这里仅介绍与RK628F/H相关的调试问题

7.1 黑屏问题排查

调试过程中，最常见的问题就是打开APK黑屏不显示，这边主要介绍一下黑屏问题的出现的可能以及排查定位问题的主要方式，黑屏问题可以先依照如下case进行排查。

1. 正常打开数据流log

APK打开黑屏可以先查看串口log，是否有正常开流，正常开流的log参考如下，驱动会调用到csi2_dphy_s_stream、rk628_csi_s_stream等接口使能数据流。

```
[  T549] m00_b_rk628-csi 5-0050: user set color range: 0
[  T549] rk628-csi-v4l2 5-0050: csc process mode: RGB L->YUV709 F
[  T549] rkcif-mipi-lvds1: stream[0] start streaming
[  T549] rkcif-mipi-lvds1: Allocate dummy buffer, size: 0x010e0000
[  T549] rockchip-mipi-csi2 mipi1-csi2: stream on, src_sd: 00000000fe8c128f,
sd_name:rockchip-csi2-dphy0
[  T549] rockchip-mipi-csi2 mipi1-csi2: stream ON
[  T549] rockchip-csi2-dphy0: dphy0, data_rate_mbps 1300
[  T549] rockchip-csi2-dphy csi2-dphy0: csi2_dphy_s_stream stream on:1, dphy3,
ret 0
[  T549] m00_b_rk628-csi 5-0050: dual mipi mode, word count user define: 3840
[  T549] rk628-csi-v4l2 5-0050: src 3840x2160 clock:593965250
[  T549] rk628-csi-v4l2 5-0050: dst 3840x2160 clock:593965250
[  T549] rk628-csi-v4l2 5-0050: mipi dphy0 hs config, manual: true
[  T549] rk628-csi-v4l2 5-0050: mipi dphy1 hs config, manual: true
[  T549] rk628-csi-v4l2 5-0050: csc process mode: RGB L->YUV709 F
[  T549] m00_b_rk628-csi 5-0050: rk628_csi_s_stream: on: 1, 3840x2160@60
[  T549] m00_b_rk628-csi 5-0050: rk628f detect pixclk more than 300M, use dual
mipi mode
```

2. HAL层dump图像

出现黑屏问题，首先要定位是应用端的问题还是驱动没有数据，或者图像数据就是黑的。应用端可以通过dump图像的方式抓取应用从驱动获取的图像。可以在黑屏的场景dump图像：

TV方式dump图像参考6.10，camera方式dump图像参考6.12

如果是dump出来的图像是黑的，则说明驱动给的数据就是黑的，如果dump的文件夹为空，则可能驱动没有输出数据流。

3. 驱动是否正常输出数据流。

参考6.1章节打开TV或者camera的debug开关，如果数据流正常的话，log会正常打印QBUF DQBUF的流程，类似如下所示：

```
E tv_input_HinDevImpl: workThread:line 2147 | start VIDIOC_DQBUF
.......
E tv_input_HinDevImpl: qBuf:line 2487 | VIDIOC_QBUF index=3, fd=21 successful
```

4. 驱动使用v4l2抓数据

如果是必现的问题，可以不跑应用，直接使用v4l2抓取数据输出，查看驱动直接抓的数据是否异常。参考6.9。

5. TV报错找不到设备

若采用的是TV的预览方式，logcat报错出现如下报错，可能会引起黑屏问题：

```
E tv_input: hinDevImpl->findDevice 0 failed!
```

可以先根据log查看TV相关的属性值是否正确，常见属性值错误如下，读取的mHdmiInType为0，rk628是MIPI通路，该值需要为1才正确，该值错误可检查tvinput.hdmiin.type=1属性是否配置，尤其是使用RK3576、RK3588S等不自带HDMIRX模块或者DTS关闭HDMIRX模块的场景，该属性值需要写入到device配置中，参考4.2章节描述。

```
E tv_input_HinDevImpl: HinDevImpl:line 187 | soc ro.board.platform=rk3576
E tv_input_HinDevImpl: prop value : mHdmiInType=0, mDebugLevel=0, mSkipFrame=0,
mPcieMode=0
```

若mHdmiInType的属性值正确，但是TV还是报错找不到设备，参考log如下。该情况，dts配置的module name:为HDMI-MIPI，但是链接的是rkcif-mipi-lvds1，index不对应，导致设备打开失败，需要检查dts中module name的配置，可详细参考3.1章节的说明。

```
E tv_input_HinDevImpl: HinDevImpl:line 187 | soc ro.board.platform=rk3576
E tv_input_HinDevImpl: prop value : mHdmiInType=1, mDebugLevel=0, mSkipFrame=0,
mPcieMode=0
......
D tv_input_HinDevImpl:  v4l device v4l-subdev2 found
I tv_input_HinDevImpl: findDevice:line 477 | findDevice open device /dev/v4l-
subdev2 successful.
E tv_input_HinDevImpl: sensor name: rk628-csi, module name: HDMI-MIPI
E tv_input_HinDevImpl: csiNum=
......
E tv_input_HinDevImpl: VIDIOC_QUERYCAP /dev/video2 cap.bus_info=platform:rkcif-
mipi-lvds1
E tv_input_HinDevImpl: findDevice:line 566 | [findDevice 566]
mHinDevHandle:ffffffff mHinDevEventHandle:9
E tv_input: hinDevImpl->findDevice 0 failed!
```

6. HDCP导致黑屏问题

如果HAL排查数据流的流程正常，但是抓取的图像是黑的，使用v4l2抓取的图像数据同样也是黑的，即可以确认是RK628输出的就是黑的，可先排查HDMI信号源是否带HDCP，如果信源带HDCP，RK628没烧写key则会导致黑屏。常见的场景是信源为DVD、部分电视盒子等设备。

出现此种异常，可以烧写HDCP的key进行排查解决。

7. AVMUTE导致异常

同上述，如果确定是RK628输出的数据异常，可以查看对应的寄存器是否是avmute导致的黑屏，可以抓取hdmirx的寄存器，查看0x30380寄存器是否异常，如出现如下bit2为1，则是异常。

```
0x30380: 00000002
```

参考如下修改解决：

```
diff --git a/drivers/media/i2c/rk628/rk628_bt1120_v4l2.c
b/drivers/media/i2c/rk628/rk628_bt1120_v4l2.c
index 4e24bc4f680b..601b726af330 100644
--- a/drivers/media/i2c/rk628/rk628_bt1120_v4l2.c
+++ b/drivers/media/i2c/rk628/rk628_bt1120_v4l2.c
@@ -617,6 +617,11 @@ static void enable_stream(struct v4l2_subdev *sd, bool en)
                }
                rk628_hdmirx_vid_enable(sd, true);
                enable_bt1120tx(sd);
+
+               rk628_i2c_update_bits(bt1120->rk628, HDMI_RX_PDEC_CTRL,
+                                     GCPFORCE_CLRAVMUTE_MASK,
GCPFORCE_CLRAVMUTE(1));
+               rk628_i2c_update_bits(bt1120->rk628, HDMI_RX_PDEC_CTRL,
+                                     GCPFORCE_CLRAVMUTE_MASK,
GCPFORCE_CLRAVMUTE(0));
        } else {
                rk628_i2c_write(bt1120->rk628, GRF_SCALER_CON0, SCL_EN(0));
                rk628_hdmirx_vid_enable(sd, false);
diff --git a/drivers/media/i2c/rk628/rk628_csi_v4l2.c
b/drivers/media/i2c/rk628/rk628_csi_v4l2.c
index dcd060ee4106..fd7b38b6cd99 100644
--- a/drivers/media/i2c/rk628/rk628_csi_v4l2.c
+++ b/drivers/media/i2c/rk628/rk628_csi_v4l2.c
@@ -911,6 +911,11 @@ static void enable_stream(struct v4l2_subdev *sd, bool en)
                        rk628_csi_enable_csi_interrupts(sd, true);
                }
                rk628_hdmirx_vid_enable(sd, true);
+
+               rk628_i2c_update_bits(csi->rk628, HDMI_RX_PDEC_CTRL,
+                                     GCPFORCE_CLRAVMUTE_MASK,
GCPFORCE_CLRAVMUTE(1));
+               rk628_i2c_update_bits(csi->rk628, HDMI_RX_PDEC_CTRL,
+                                     GCPFORCE_CLRAVMUTE_MASK,
GCPFORCE_CLRAVMUTE(0));
        } else {
                if (csi->plat_data->tx_mode == CSI_MODE) {
                        rk628_csi_enable_csi_interrupts(sd, false);
diff --git a/drivers/media/i2c/rk628/rk628_hdmirx.h
b/drivers/media/i2c/rk628/rk628_hdmirx.h
index 015c3df0aa38..76a81380dff2 100644
--- a/drivers/media/i2c/rk628/rk628_hdmirx.h
```

```
+++ b/drivers/media/i2c/rk628/rk628_hdmirx.h
@@ -243,6 +243,8 @@
 #define PFIFO_STORE_GCP(x)              UPDATE(x, 17, 17)
 #define PFIFO_STORE_ACR_MASK            BIT(16)
 #define PFIFO_STORE_ACR(x)              UPDATE(x, 16, 16)
+#define GCPFORCE_CLRAVMUTE_MASK                 BIT(14)
+#define GCPFORCE_CLRAVMUTE(x)           UPDATE(x, 14, 14)
 #define GCPFORCE_SETAVMUTE_MASK                 BIT(13)
 #define GCPFORCE_SETAVMUTE(x)           UPDATE(x, 13, 13)
 #define PDEC_BCH_EN_MASK                BIT(0)
@@ -258,6 +260,8 @@
 #define DVI_DET                                 BIT(28)
 #define HDMI_RX_PDEC_GCP_AVMUTE                 (HDMI_RX_BASE + 0x0380)
 #define PKTDEC_GCP_CD_MASK              GENMASK(7, 4)
+#define PKTDEC_GCP_SETAVMUTE_MASK       GENMASK(1, 1)
+#define PKTDEC_GCP_CLRAVMUTE_MASK       GENMASK(0, 0)
 #define HDMI_RX_PDEC_AVI_HB             (HDMI_RX_BASE + 0x03a0)
 #define HDMI_RX_PDEC_AVI_PB             (HDMI_RX_BASE + 0x03a4)
 #define VID_IDENT_CODE_VIC7             BIT(31)
```

8. MIPI CRC相关报错

若出现MIPI CRC等相关的报错，可以参考《Rockchip_Developer_Guide_Android12+_HDMI_IN_Bridge_CN》，排查MIPI信号是否存在干扰、硬件走线等情况。

9. 其他相关

若排查不属于上述的范围，则可以抓取完整的log，提供至redmine，请求RK的协助。完整的log包括：dump的图像源文件，打开TV HAL/cameraHAL debug的 locgat，打开rk628驱动log开关以及vicap驱动log开关的串口log。

7.2 4096x2160分辨率异常问题

接RK3576平台时，4096x2160分辨率的源数据会概率性出现异常，会出现iommu的报错，导致不可恢复，可以增加如下修改，在support_mode增加最大分辨率到4096x2160.

```
@@ -274,6 +274,15 @@ static struct rkmodule_csi_dphy_param rk3588_dcphy_param = {

 static const struct rk628_csi_mode supported_modes[] = {
        {
+               .width = 4096,
+               .height = 2160,
+               .max_fps = {
+                       .numerator = 10000,
+                       .denominator = 600000,
+               },
+               .hts_def = 4400,
+               .vts_def = 2250,
+       }, {
                .width = 3840,
                .height = 2160,
                .max_fps = {
```

7.3 信号源1440x240黑屏

因为HDMIIN分配的是Esmart图层，最大只能支持8倍的缩放，超过8倍的话无法支持，所以需要确认以下几点：

1. 需要输出接口的分辨率，`cat /d/dri/0/summary`，确认是否是4K或是更大分辨率输出，如果是，那已经起来8倍。
2. 如果一定要测试这么小的分辨率，可以把输出的分辨率切小一些，比如1080P.
3. 反过来也是一样的原理，比如：IN是4K，OUT是1440x240，缩小倍数超过了8倍，一样是不支持的。

7.4 1366x768分辨率异常

RK3576/RK3588 搭配RK628F实现HDMIIN功能，使用的是RK3588/RK3576 VICAP控制器接收图像，VICAP控制器对输入图像数据有限制，必须满足宽8对齐的要求，才能正常接收，1366分辨率不满足8对齐，因此可能会出现异常，一般建议将不满足8对齐的分辨率向下裁减至8对齐，也就是1366x768可裁减成1360x768，每一行会丢失6个像素。驱动可直接在get_fmt设置上报裁减后的分辨率，框架会自动裁剪：

```
format->format.width = ALIGN_DOWN(csi->timings.bt.width, 8);
```

若是选择走的camera框架，对应的camera3_profiles.xml中也必须是添加裁减后的分辨率。

7.5 应用接口使用说明

RK628F/H驱动提供部分接口供应用查询输入源的分辨率格式等信息，如应用场景需要，可以使用响应的ioctl调用获取驱动信息，目前已实现的ioctl接口包括如下：

- VIDIOC_SUBDEV_QUERY_DV_TIMINGS：查询信号源输入分辨率信息
- VIDIOC_G_CTRL：获取5V插入状态
- VIDIOC_SUBDEV_G_FMT：获取MIPI输出分辨率
- RK_HDMIRX_CMD_GET_SIGNAL_STABLE_STATUS：获取信号输入源锁定状态
- RK_HDMIRX_CMD_GET_FPS：获取信号源输入帧率信息
- RK_HDMIRX_CMD_GET_INPUT_MODE：获取输入源HDMI/DVI模式
- RK_HDMIRX_CMD_GET_COLOR_RANGE：获取输入源的色域信息
- RK_HDMIRX_CMD_GET_COLOR_SPACE：获取输入源的颜色空间
- RK_HDMIRX_CMD_SET_COLOR_RANGE：设置RK628F/H色域转换，效果可参考3.9色域转换说明章节

应用编程使用上述接口时，需要获取RK628F/H的设备节点，一般是/dev/v4l-subdevX，可以通过如下命令查看：

```
grep -H '' /sys/class/video4linux/v4l-subdev*/name
```

应用编程示例可参考如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <getopt.h>
#include <fcntl.h>
#include <unistd.h>
```

```
#include <sys/ioctl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <dirent.h>
#include <string.h>
#include <linux/videodev2.h>
#include <linux/v4l2-subdev.h>

#define RKMODULE_GET_HDMI_MODE       \
    _IOR('V', BASE_VIDIOC_PRIVATE + 34, __u32)

#define RK_HDMIRX_CMD_GET_FPS \
    _IOR('V', BASE_VIDIOC_PRIVATE + 0, int)

#define RK_HDMIRX_CMD_GET_SIGNAL_STABLE_STATUS \
    _IOR('V', BASE_VIDIOC_PRIVATE + 1, int)

#define RK_HDMIRX_CMD_GET_COLOR_SPACE \
    _IOR('V', BASE_VIDIOC_PRIVATE + 10, int)

const int kMaxDevicePathLen = 256;
const char* kDevicePath = "/dev/";
const char kPrefix[] = "v4l-subdev";
const int kPrefixLen = sizeof(kPrefix) - 1;
const int kDevicePrefixLen = sizeof(kDevicePath) + kPrefixLen + 1;
char kV4l2DevicePath[kMaxDevicePathLen];
int mipifd = 0;

static const char *bus_color_space_str[8] = {
    "xvYCC601", "xvYCC709", "sYCC601", "Adobe_YCC601",
    "Adobe_RGB", "BT2020_YcCbcCrc", "BT2020_RGB_OR_YCbCr", "RGB"
};

int find_mipi_fd()
{
        DIR *devdir;
        struct dirent* de;
        int videofd;
        int ret;

        devdir = opendir(kDevicePath);
        if(devdir == 0) {
                printf("%s: cannot open %s!\n", __FUNCTION__, kDevicePath);
                return -1;
        }

        while ((de = readdir(devdir)) != 0) {
                if (!strncmp(kPrefix, de->d_name, kPrefixLen)) {
                        printf("found %s\n", de->d_name);
                        snprintf(kV4l2DevicePath, kMaxDevicePathLen,"%s%s",
kDevicePath, de->d_name);
                        videofd = open(kV4l2DevicePath, O_RDWR);
                        if (videofd < 0){
                                printf("[%s %d] open device failed:%x\n",
__FUNCTION__, __LINE__, videofd);
                                continue;
                        } else {
                                uint32_t ishdmi;
```

```
                                ret = ioctl(videofd, RKMODULE_GET_HDMI_MODE,
(void*)&ishdmi);
                                if (ret < 0) {
                                        printf("RKMODULE_GET_HDMI_MODE
Failed\n");
                                        close(videofd);
                                        continue;
                                }
                                printf("%s
RKMODULE_GET_HDMI_MODE:%d\n",kV4l2DevicePath,ishdmi);
                                if (ishdmi) {
                                        mipifd = videofd;
                                        printf("MipiHdmi fd:%d\n",mipifd);
                                        if (mipifd < 0) {
                                                return -1;
                                        }
                                }
                        }
                }
        }
        closedir(devdir);
        return ret;
}

int main()
{
        int ret;
        int fd = -1;

        find_mipi_fd();
        fd = mipifd;
        if (fd < 0) {
                printf("find mipi dev fail\n");
                return -1;
        }

        //查询拔插状态
        struct v4l2_control control;
        control.id = V4L2_CID_DV_RX_POWER_PRESENT;
        ret = ioctl(fd, VIDIOC_G_CTRL, &control);
        if (ret < 0) {
                printf("Set POWER_PRESENT failed ,%d\n", ret);
        }

        if (control.value == 1) {
                //获取信号状态与分辨率
                int signal_status = 0;
                int ret = ioctl(fd, RK_HDMIRX_CMD_GET_SIGNAL_STABLE_STATUS,
&signal_status);
                if (ret < 0)
                        printf("RK_HDMIRX_CMD_GET_SIGNAL_STABLE_STATUS failed:
%d\n", ret);

                printf("get signal status:%d\n", signal_status);
                struct v4l2_dv_timings timings;
                int err = ioctl(fd, VIDIOC_SUBDEV_QUERY_DV_TIMINGS, &timings);
                if (ret < 0)
```

```
                        printf("VIDIOC_SUBDEV_QUERY_DV_TIMINGS failed: %d\n",
ret);
                const struct v4l2_bt_timings *bt = &timings.bt;

                printf("hact:%d, hfp:%d, hs:%d, hbp:%d, vact:%d, vfp:%d, vs:%d,
vbp:%d, interlace:%d\n",
         bt->width, bt->hfrontporch, bt->hsync, bt->hbackporch, bt->height, bt-
>vfrontporch, bt->vsync,
         bt->vbackporch, bt->interlaced);

                struct v4l2_subdev_format aFormat;
                err = ioctl(fd, VIDIOC_SUBDEV_G_FMT, &aFormat);
                if (err < 0)
                        printf("VIDIOC_SUBDEV_G_FMT failed: %d\n", err);

                int fps;
                err = ioctl(fd, RK_HDMIRX_CMD_GET_FPS, &fps);
                if (err < 0)
                        printf("RK_HDMIRX_CMD_GET_FPS failed: %d\n", err);
                printf("VIDIOC_SUBDEV_G_FMT: width: %d, height: %d, fps: %d\n",
aFormat.format.width,aFormat.format.height, fps);

                int color_space = 0;
                ret = ioctl(fd, RK_HDMIRX_CMD_GET_COLOR_SPACE, &color_space);
                if (ret < 0)
                        printf("RK_HDMIRX_CMD_GET_COLOR_SPACE failed: %d\n",
ret);

                printf("get color space:%s\n", bus_color_space_str[color_space]);
        } else {
                printf("get 5V:%d, hdmi plug out\n", control.value);
        }

        close(fd);
        return 0;
}
```

7.6 闪花屏问题说明

RK628F/H在对接RK3588/RK3576主控实现HDMI-IN的功能时，概率出现拔插或者切分辨率闪一帧花屏的问题，针对该问题的，文档统一说明如下：

问题现象描述：在RK628F输入HDMI的信号发生变化时（拔插或者切换分辨率、切换信号源通道等操作），会出现短暂的花屏帧，如下图所示。



问题原因分析：RK628F输入的HDMI信号发生变化时，在变化时间内HDMI是没有数据流的状态，内部缓存区数据会停留在切换之前的数据，但此时RK628F的输出还是正常打开，因此缓存区给到输出端的数据就是重复数据，最后输出就会出现类似上图的一段重复数据的花帧。

目前优化思路：对于RK628F端，需要尽快识别到前级信号的变化，在最快的时间内将缓存区数据设置为纯黑或者关闭输出。对接RK3588/RK3576主控的时候，RK628F是一个i2c设备，中断响应都是通过gpio进行，因此需要尽可能优化gpio的响应速度。

对于主控端，需要帧完整校验机制，若收到的帧不完整，则直接丢帧，可以一定概率上丢掉该异常帧。RK3588/RK3576/RK3572 默认会有帧完整检验，但如果是打开了低延时提前送帧，则无法检查帧完整。

以下针对花屏的几个常见场景，分别说明优化的思路：

场景一：外部信源拔插：预览过程拔插信源，出现最后一帧闪花屏的现象（花屏帧一闪而过）。

注：场景为拔出后闪一帧花屏后即黑屏，若一直停留在花屏帧，则需要更新代码版本解决问题。

优化思路：主控端尽量快识别到RK628F前级信号源的拔插动作，并操作RK628F第一时间关闭输出数据流。软件流程最新代码已经合并，GPIO拔插响应还需要依赖硬件响应，即硬件上尽量优化HDMI端口5V掉电的速度。实际测试中可以将HDMI的5V端口、拔插的GPIO、RK628F输出的MIPI数据流三者同时接入示波器查看响应速度，尽可能优化5V变化到GPIO变化的时间。

场景二：外部信源切换分辨率：预览过程切换信源分辨率或者信号，出现闪花屏而后正常出图。

优化思路：识别到信号变化的中断后第一时间关闭数据流或者设置缓存区为纯黑数据，但RK628F是i2c设备外设，信号变化需要RK628F内部先识别再通过GPIO中断通知主控，实际优化效果有限，目前没有其他优化手段。

场景三：外部信源切换通道：在带有switch的场景下，外部信源发起切换通道，这类场景类似拔插的流程。

优化思路：同拔插的方式。

场景四：主控主动切换信源通道：在有switch的场景下，switch前级的信号同时接入多个信号源，在SOC的端，app去操作switch切换信源通道，切换过程出现闪花屏现象。

优化思路：该场景为主控主动切换的场景，在SOC端可以在切换之前主动关闭数据流，或者通过i2c将RK628F的缓存区设置为纯黑色的数据，之后再去设置switch的通道，可以规避这个闪屏现象。

RK628F驱动代码提供关闭输出数据流接口：

DSI通路：rk628_dsi_disable

CSI通路：rk628_csi_disable_stream

RK628F驱动代码提供设置缓存区纯黑接口：rk628_set_bg_enable

客户可以根据需求，自行将上述接口封装引出给应用层进行调用。